

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Investigación: Formulario contenedor

Programación II

Saimon Daniel Gonzalez López

9989-25-10617



Investigación Técnica: Arquitectura de Interfaz MDI en Java Swing

Formulario Contenedor MDI (Multiple Document Interface)

Un formulario contenedor MDI (Multiple Document Interface o Interfaz de Múltiples Documentos) es un patrón de diseño gráfico en aplicaciones de escritorio donde una ventana principal actúa como marco o contenedor para albergar múltiples ventanas secundarias (formularios hijos). A diferencia de las aplicaciones SDI (Single Document Interface), donde cada ventana opera de forma independiente en la barra de tareas del sistema operativo, el patrón MDI centraliza el flujo de trabajo dentro de un único espacio delimitado.

Componentes Fundamentales en Java Swing

¿Qué es un JDesktopPane?

El JDesktopPane es un contenedor especializado de Java Swing que simula un "escritorio virtual" dentro de un marco principal (JFrame). Hereda de la clase JLayeredPane y su función técnica es gestionar la representación, apilamiento (capas Z-order) y posicionamiento de las ventanas internas que se abren dentro de él.

¿Qué es un JInternalFrame?

Un JInternalFrame representa una ventana secundaria o "hija" diseñada para desplegarse exclusivamente dentro de un JDesktopPane. A diferencia de un JFrame convencional, un JInternalFrame no es una ventana nativa del sistema operativo, sino un componente administrado por Swing que incluye capacidades integradas como maximizar, minimizar, redimensionar y cerrar dentro del escritorio virtual.

¿Cómo funciona un JMenuBar?

El JMenuBar es la barra de menú superior anclada al formulario contenedor principal. Su funcionamiento se basa en una jerarquía de componentes:

JMenuBar: La barra contenedora horizontal.

JMenu: Las opciones principales desplegables (Ej. "Archivo", "Ventas", "Reportes").

JMenuItem: Las opciones finales seleccionables que disparan eventos de acción (ActionListener) al ser presionadas por el usuario.

Apertura de Formularios Hijos desde un Menú

Para instanciar y mostrar un formulario hijo (JInternalFrame) dentro del escritorio virtual (JDesktopPane) mediante un clic de menú, se registra un listener en el JMenuItem correspondiente. A continuación se presenta la estructura de código estándar:

```

import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.beans.PropertyVetoException;
// Formulario Principal MDI Container
public class MainMDIFrame extends JFrame {
    private JDesktopPane desktopPane;
    private JMenuBar menuBar;
    private JMenu menuVentas;
    private JMenuItem itemNuevaFactura;
    public MainMDIFrame() {
        setTitle("Sistema - MDI Container");
        setSize(1024, 768);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // 1. Inicializar el escritorio virtual y asignarlo como contenedor principal
        desktopPane = new JDesktopPane();
        this.setContentPane(desktopPane);
        // 2. Construcción de la barra de menú
        menuBar = new JMenuBar();
        menuVentas = new JMenu("Ventas");
        itemNuevaFactura = new JMenuItem("Nueva Factura");
        menuVentas.add(itemNuevaFactura);
        menuBar.add(menuVentas);
        this.setJMenuBar(menuBar);
        // 3. Evento para abrir el formulario hijo desde el menú
        itemNuevaFactura.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                abrirFormularioFactura();
            }
        });
    }
}

```

```

    }
    private void abrirFormularioFactura() {
        // Instanciar el JInternalFrame (Formulario Hijo)
        InvoiceInternalForm invoiceForm = new InvoiceInternalForm();
        // Agregar al JDesktopPane y hacerlo visible
        desktopPane.add(invoiceForm);
        invoiceForm.setVisible(true);
        try {
            // Dar el enfoque activo al formulario recién abierto
            invoiceForm.setSelected(true);
        } catch (PropertyVetoException ex) {
            System.err.println("Error al enfocar el formulario: " + ex.getMessage());
        }
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new MainMDIFrame().setVisible(true);
    });
}

// Clase que representa el Formulario Hijo (JInternalFrame)
class InvoiceInternalForm extends JInternalFrame {
    public InvoiceInternalForm() {
        super("Formulario Hijo", true, true, true, true); // (título, resizable, closable,
maximizable, iconifiable)
        setSize(600, 400);
    }
}

```

Ventajas de la Arquitectura MDI en Aplicaciones de Escritorio

Organización Centralizada

Mantiene todas las herramientas e interfaces secundarias ordenadas dentro de una sola ventana de trabajo, evitando la dispersión de ventanas en la barra de tareas del sistema operativo.

Navegación Eficiente

Facilita el acceso estructurado a múltiples módulos del sistema mediante una barra de menú superior siempre accesible.

Gestión Eficiente de Recursos

Permite reutilizar instancias de componentes y mantener estados compartidos a través del formulario padre sin reabrir múltiples sesiones.

Control de Entorno (UI Consistente)

Brinda al usuario un entorno de trabajo unificado y profesional, ideal para sistemas empresariales o de gestión (ERP, POS, CRM).

Bibliografía y Fuentes Consultadas

Oracle. (n.d.). How to Use Internal Frames. The Java™ Tutorials.
<https://docs.oracle.com/javase/tutorial/uiswing/components/internalframe.html>

Oracle. (n.d.). How to Use Menus. The Java™ Tutorials.
<https://docs.oracle.com/javase/tutorial/uiswing/components/menu.html>

Schildt, H. (2018). Java: The Complete Reference (11th ed.). McGraw-Hill Education.